

DutchBay Feasibility Report — Module Coverage Analysis

Which of the 294 source modules the **Complete Feasibility Study** (2026-07-07, wind, engine v15.3.0 @ a50b0bfce8e8) actually exercised — and, for every module it did **not**, the concrete reason.

1. Summary

Coverage	Count	Meaning
● Fired	92	code executed while producing the deliverable
● Available, not fired	108	feasibility-relevant, could contribute, but did not run this pass
○ Not applicable	94	not part of a wind feasibility deliverable at all
Total	294	

Read: ~31% of the codebase fired for this deliverable. That is expected and correct — the model is a broad platform (web service, BESS track, grid studies, a full sensitivity/optimization suite, CI tooling) and a single flat-LKR wind lender case exercises the finance spine + the wind-resource spine + three report emitters, while the rest is either gated off, superseded by committed inputs, or out of scope. The two buckets below say exactly which and why.

Two evidence-over-manifest reconciliations (the cataloguing agents read source that corrected the run manifest): `finance/bess_revenue.py` is classified **fired** — the wind cashflow imports and calls it every run as a byte-identical no-op (returns `None/0.0` with no `type: bess block`); `finance/epc_margin.py` is **not applicable** — it is only imported by a standalone Kolonnawa BESS/EPC CLI, not the operational cashflow (the manifest had conflated it with `epc_helper_v14`, which *is* fired).

2. Why modules were NOT included (the direct answer)

2a. Available but not fired — feasibility-relevant, gated or superseded

Standalone sensitivity / MC / optimization (not run as steps) — 31 modules. `capital_structure_optimizer_v14.py`, `__init__.py`, `cli_capital_structure_optimize_hydra.py`, `cli_monte_carlo_hydra.py`, `cli_sensitivity_hydra.py`, `sensitivity_runner.py`, `mc_capex.py`, `dscr_sensitivity.py`, `fx_sensitivity_real.py`, `monte_carlo_v14.py`, `optimization_v14.py`, `__init__.py`, `adapters.py`, `docstrings.py`...

Manifest lists `capital_structure_optimizer_v14` explicitly under 'standalone sensitivity/optimization NOT run as separate steps' — it is opt-in and only invoked via its dedicated `analytics/cli/cli_capital_structure_optimize_hydra.py` CLI, which the lender feasibility run did not call (risk was covered by the 2000-trial capital-risk MC + deterministic 8-scenario range).

Grid capability (advisory, default-off gate) — 23 modules. `__init__.py`, `__init__.py`, `solar.py`, `wind.py`, `curtailment_qsts.py`, `evaluate_grid.py`, `frequency_response.py`, `grid_interface_schema.py`, `harmonics.py`, `__init__.py`, `frequency_response.py`, `poc_aggregation.py`, `reactive_screen.py`, `ride_through.py`...

Manifest names the ENTIRE `analytics/grid/*` capability as advisory default-off; the lender scenario sets `grid.study_enabled=false` (master gate), so this package's screens were never invoked during the wind feasibility deliverable.

Upstream wind stack (bypassed by committed AEP summary) — 19 modules. `aep_provenance.py`, `aep_reconciliation.py`, `monte_carlo_aep.py`, `aep_summary_builder.py`, `mc_aep_weibull.py`, `pipeline_aep_v14.py`, `siting_metadata.py`, `wind_integration.py`, `wind_rose.py`, `wind_rose_plot.py`, `arco_assessment.py`, `__init__.py`, `crossval.py`, `era5_fetcher.py`...

Manifest explicitly lists `analytics/aep_provenance` among the 'wind modules NOT fired this pass' (grouped with `aep_reconciliation`); finance used the committed `aep_summary_dutchbay_10mw.json` rather than re-loading an authored scenario declaring `resource.power_curve.source_id`, so this load-time provenance guard was not triggered.

Solar-resource detail (core touched only via the hybrid/solar tech-comparison) — 8 modules. `bifacial_guard.py`, `cashflow_adapter.py`, `exceedance.py`, `long_term_trend.py`, `loss_model.py`, `pv_producer.py`, `soiling_profile.py`, `source_quality.py`

Module imported via the `solar_resource` package (`cashflow_adapter` imports it), but `assert_monofacial_financed_cf` only executes inside `solar_export_to_scenario_patch`, which the W4 (#614) solar-ingestion path is off for (no `solar_export_path`; committed hybrid/solar scenarios carry a pre-baked frozen CF 0.1685). No bifacial marker is present. (inferred)

GIS terrain/suitability layers (need external rasters) — 7 modules. `boundary_clip.py`, `dem_ingest.py`, `exclusion_mask.py`, `gwa_ingest.py`, `landcover_roughness.py`, `mcdm_suitability.py`, `rix.py`

Manifest 'GIS layers NOT fired' explicitly lists `analytics/gis/boundary_clip`; the ERA5-grid-only export produced full-grid rasters and no project-boundary polygon clip was requested this pass.

CASPER MC facet (capital-risk did not route through it) — 3 modules. `__init__.py`, `casper_payload.py`, `casper_v14.py`

Manifest flags the CASPER MC facet as fired ONLY if the capital-risk emitter routes through it; verified `capital_risk_emit.py` / `capital_risk_layer_v14.py` do NOT import `analytics.casper` (only `contracts_v14` comments, `sensitivity/tail_risk` (not fired), and `mc/covenant` reference it), so the package was not imported during the study run.

Package facade / re-export (not on the feasibility import path) — 3 modules. `__init__.py`, `__init__.py`, `__init__.py`

Manifest names `contracts_v14` as fired, but the fired modules import from `analytics.contracts_v14` directly; this facade package is only exercised if a caller imports via `analytics.contracts`, which the feasibility path does not do. (inferred)

Other feasibility-relevant, not fired — 3 modules. `parameter_solvers.py`, `exports.py`, `cashflow_adapter.py`

Docstring states these are read-only analyst tools with NO committed-scenario caller and no Hydra CLI wiring (#615 deferred); the feasibility pipeline run does not invoke any solver, so they did not fire.

Cost engine (fixed committed capex; benchmark/QRA not re-run) — 3 modules. `benchmark.py`, `cost_basis.py`, `estimate_class.py`

Its functions did not run on the wind study: `lcos_benchmark` is invoked only inside `scenario_analytics`' `compute_lcos_suite` (BESS/storage LCOS, empty for a pure-wind scenario), and `capex_benchmark` is imported only by the `not_applicable` web layer `api/pipeline_api.py`; module import alone (via `scenario_analytics`) did not exercise the benchmark logic (inferred).

FX calibration/history (pinned FX vintage; no live calibration) — 2 modules. `fx_calibration.py`, `fx_history.py`

Opt-in: `analytics.mc.engine` only initializes it when a `distribution: fx_calibrated` MC param is declared; the fired capital-risk MC uses the lender scenario's plain-`uniform` `fx` driver (parameter list [`capacity_factor`, `tariff`, `opex`, `capex`, `fx`, `curtailment`], no `fx_calibrated`), so `calibrate_from_config` was never reached, and `fx_sensitivity_real` / full-population MC did not fire.

Schema modules (not in strict validate set) — 2 modules. `era5_interface_schema.py`, `wind_interface_schema.py`

`schema_guard` imports+registers the 'era5' module only when 'era5' is in the `validate` list, but the finance run validates the default ['cashflow', 'debt']; the ERA5 retrieval that DID fire went through `wind_resource.era5_retrieval/ERA5RequestConfig` directly, not this `schema-guard` path. Not named in the manifest's fired list (inferred).

Alternate entry point (not the path the study used) — 2 modules. `run_scenario_analytics_v14.py`, `run_wind_analysis_v14.py`

The feasibility deliverable used the canonical `run_full_pipeline_v14.py` path (manifest 'What FIRED'); this deliberately-lighter batch-compare CLI (PIPE-1) was not the run entrypoint and did not fire this pass (inferred).

Portfolio/curtailment detail (single-tech wind lender case) — 1 modules. `poi_curtailment.py`

Opt-in: `resolve_shared_poi_curtailment` returns `None` unless a scenario declares `generation.shared_poi.limit_mw` AND ≥ 2 techs supply `generation.technologies.<tech>.hourly_profile_mw` (hourly TMY/wind profiles tracked in #529, not present). The manifest also states DutchBay's 220kV line is a separate CEB project so the POI is not binding, and the emitter/portfolio fired list does not name it. (inferred)

Interaction-grid emitter (no config block) — 1 modules. `interaction.py`

The `emit_interaction_grid` emitter is gated off — the lender scenario declares no `interaction_grid` metric/param block and the emitter fail-louds — so `analytics/sensitivity/interaction` did NOT run (manifest line 31).

2b. Not applicable — outside a wind feasibility deliverable

Tooling / CI / research scripts — 47 modules. `01_github_repo_scanner.py`, `FINAL_CORRECTED_sensitivity_v14.py`, `FIX_SENSITIVITY_LINE_489.py`, `add_pydantic_v2_compat_stubs.py`, `analyze_directory.py`, `gen_scenario_yaml.py`,

wacc_engine_yaml.py, build_zip_from_manifest.py, generate_manifest.py, make_essential_zip.py, check_fields.py, check_all_py_files.py, check_legacy_imports.py, check_staged_py_files.py...

Repo-inventory/tech-debt tooling, not a feasibility module; manifest excludes non-feasibility scripts/tooling (inferred).

Web-service layer (#788) — 29 modules. __init__.py, path_safety.py, pipeline_api.py, sensitivity_api.py, __init__.py, __init__.py, auth.py, config.py, jobs_router.py, main.py, responses.py, surface.py, __init__.py, config.py...

Manifest line 40 lists top-level api/ among the FastAPI web-service productization (#788) that is not exercised by a study run.

Legacy / sandbox / examples — 10 modules. __init__.py, __init__.py, pysam_runner.py, dashboard_demo.py, __init__.py, dutchbay_bootstrap.py, monte_carlo_lender_pack_example.py, __init__.py, stress_tests_v14.py, make_clean_zip.py

Manifest lists analysis_tools/* under 'Legacy / sandbox / examples' not part of a wind feasibility deliverable; the package doc itself says 'NOT for production pipelines - use analytics/ instead', so it was never imported by the feasibility run (no pipeline/report caller).

BESS track (separate workstream) — 5 modules. bess.py, bess_soc.py, bess_lcos.py, bess_project_economics.py, epc_margin.py

Manifest lists analytics/grid/capabilities/bess* explicitly under not_applicable (separate BESS track, not this wind study); also under the default-off grid gate.

Other non-feasibility — 3 modules. cli_sensitivity.py, streamlit_app.py, dutchbay_bootstrap_rules.py

Deprecated argparse shim (removal planned Sprint 18) superseded by cli_sensitivity_hydra.py; not part of the wind-feasibility run, which used run_full_pipeline_v14.py. (inferred)

3. Coverage by subsystem

Subsystem	Total	●	●	○
finance	23	19	1	3
finance/cashflow	1	0	1	0
finance/equity	1	0	1	0
finance/grid	2	0	2	0
analytics (top-level)	35	25	10	0
analytics/core	9	7	2	0
analytics/contracts	1	0	1	0
analytics/mc	9	8	1	0
analytics/sensitivity	13	0	12	1
analytics/wind	12	3	9	0
analytics/power_curves	1	1	0	0
analytics/loader	1	1	0	0
analytics/gis	9	2	7	0
analytics/grid	18	0	16	2
analytics/cost	6	2	4	0
analytics/fx	7	5	2	0
analytics/casper	3	0	3	0
analytics/portfolio	5	4	1	0
analytics/cli	5	0	4	1
analytics/dashboard	1	0	0	1
analytics/simulation	1	0	1	0

Subsystem	Total	●	●	○
analytics/pysam_sandbox	2	0	0	2
wind_resource	17	6	11	0
solar_resource	9	1	8	0
app	1	0	0	1
app/reports	9	6	3	0
app/api	7	0	0	7
app/jobs	8	0	0	8
app/services	4	0	0	4
app/web	3	0	0	3
app/models	2	0	0	2
api	4	0	0	4
scripts	34	0	7	27
scripts/ci	6	0	0	6
scripts/build	3	0	0	3
scripts/github	2	0	0	2
scripts/research	4	0	0	4
scripts/analysis	3	0	0	3
scripts/legacy_runners	2	0	0	2
(repo root)	5	2	1	2
legacy	2	0	0	2
legacy_scripts	1	0	0	1
examples	1	0	0	1
analysis_tools	1	0	0	1
config	1	0	0	1

4. Appendix — full coverage register

4a. Fired (92)

Module	Reason it fired
analytics/__init__.py	Package import: every fired analytics module (pipeline_v14_enhanced, evaluation_v14, contracts_v14, etc. named in the manifest 'What FIRED' list) triggers this init and its re-export chain during the lender-case pipeline run.
analytics/capital_risk_layer_v14.py	Manifest 'Emitters FIRED': emit_capital_risk_report=true (n_trials=2000) routes through analytics/capital_risk_layer_v14 + analytics/mc/*; it does NOT route through analytics/casper (verified: capital_risk_emit and this module import no casper), so the CASPER facet stays unfired while this fired.
analytics/conditions_precedent.py	Named in manifest 'What FIRED' report-only detector list (conditions_precedent); runs at load time on the lender scenario's conditions_precedent block.
analytics/config_schema.py	Named in manifest 'What FIRED' (config_schema); its registered required-field specs are consulted by the strict validation the lender-case run performs at load time.
analytics/contracts_v14.py	Named in manifest 'What FIRED'; the fired pipeline builds and serializes ScenarioResult and related contracts defined here for the lender-case output.
analytics/core/__init__.py	Package import: fired modules in the lender-case path import submodules (analytics.core.metrics via pipeline_v14_enhanced; analytics.core.risk_metrics/returns via pipeline_analytics_v14; covenant_breach via capital_risk_layer_v14/mc.exports), which imports the analytics.core package.
analytics/core/covenant_breach.py	Imported by capital_risk_layer_v14 (prob_breach) and analytics/mc/exports (prob_breach, is_floor_pinned), both on the fired 2000-trial capital-risk emitter path (emit_capital_risk_report=true) that produced the report's breach-probability blocks.

Module	Reason it fired
analytics/core/epc_helper.py	Imported by analytics.scenario_analytics (a report-only detector declared in the lender config and listed as fired); the shim routes to finance.epc_helper_v14, which the manifest lists as fired.
analytics/core/exceedance.py	Imported by wind_resource/bankable_aep (whose density/gross helpers were touched via the fired AEP tornado) and solar_resource/exceedance (fired for the hybrid/solar scenarios in the emit_tech_comparison run). (inferred)
analytics/core/metrics.py	Imported and called by analytics.pipeline_v14_enhanced.run_v14_pipeline (calculate_scenario_kpis), the finance orchestration the manifest lists as fired for the canonical lender case and 8-scenario suite.
analytics/core/returns.py	Imported by analytics.pipeline_analytics_v14 (a report-only detector declared in the lender config and listed as fired), which pulls the returns dataclasses/calculators during the canonical lender-case run. (inferred)
analytics/core/risk_metrics.py	Imported by capital_risk_layer_v14 (fired via emit_capital_risk_report=true, the 2000-trial MC that produced VaR/CVaR/breach-prob blocks) and by pipeline_analytics_v14 (fired report-only detector).
analytics/cost/__init__.py	Package import: fired debt_v14 imports analytics.cost.contingency during debt sizing, so the analytics.cost package init was imported on the deterministic scenario path (finance stack fired per manifest).
analytics/cost/contingency.py	Fired: finance/debt_v14.py imports and calls contingency_is_qra on every debt-sizing run to choose QRA-vs-fixed CAPEX (debt_v14 fired per manifest); the resolve_contingency QRA branch only recomputes when the scenario sets capex.contingency.method=qra (fixed default), but the module's guard code executed.
analytics/development_readiness.py	Named in manifest 'What FIRED' report-only detector list (development_readiness); runs at authored-scenario load time on the lender case, changing no computed number.
analytics/evaluate_scenario.py	Named explicitly in manifest 'What FIRED' orchestration list (evaluate_scenario) as part of the fired lender-case orchestration.
analytics/evaluation_v14.py	Named in manifest 'What FIRED' orchestration list; it is the evaluate_with_overrides gateway (ARCH-04) the deterministic 8-scenario suite and emitters compose through.
analytics/evidence_register.py	Named in manifest 'What FIRED' as a report-only detector declared in the lender config; runs at authored-scenario load time on the lender case.
analytics/evidence_score.py	Named in manifest 'What FIRED' report-only detector list (evidence_score); builds on the fired evidence_register during the lender-case load-time detector pass.
analytics/executive_workbook.py	Manifest states emit_executive_workbook=true -> analytics/executive_workbook -> xlsx fired during the feasibility run; it is the genuine live caller assembling the five finance frames and the ResourceTrend sheet.
analytics/export_helpers.py	ChartGenerator.plot_npv_distribution is called by analytics/capital_risk_layer_v14 (emit_npv_distribution_from_trials), which fired for the emit_capital_risk_report=true npv_distribution PNG the manifest lists as produced; the module thus executed during the feasibility deliverable.
analytics/feasibility_sections.py	Named in manifest 'What FIRED' report-only detector list (feasibility_sections); runs at load time to score the lender scenario's feasibility-section coverage.
analytics/fx/__init__.py	Package import fired: fired pipeline_v14_enhanced does from analytics.fx.fx_integration import integrate_fx_into_scenario_result and analytics/ init imports analytics.fx.fx_contracts, so the analytics.fx package was imported during the deterministic scenario run.
analytics/fx/fx_builder.py	Fired: called by fx_integration.integrate_fx_into_scenario_result, which fired pipeline_v14_enhanced invokes after the ScenarioResult is assembled to populate fx_block/fx_curve/fx_risk_profile.
analytics/fx/fx_contracts.py	Fired: imported at module load by analytics/ init and contracts_v14 (both fired) and by fx_builder; the resulting FX dataclasses are attached to the ScenarioResult produced by the deterministic run.
analytics/fx/fx_fetch.py	Fired: default_fx_lkr_per_usd / FXRequestConfig are imported and used by fired finance modules cashflow_v14_fx, cashflow_v14_params, epc_helper_v14, and analytics/scenario_loader to resolve the pinned deterministic FX rate for the scenario.
analytics/fx/fx_integration.py	Fired: fired pipeline_v14_enhanced.run_v14_pipeline calls integrate_fx_into_scenario_result (line ~955) to attach FX blocks after assembling the ScenarioResult on the deterministic run.
analytics/gis/geotiff_export.py	Imported and used by analytics/gis/gis_export (which the manifest marks FIRED) for export_grid_rasters/build_manifest_entry/append_manifest; the fired GIS export path writes rasters through this module (inferred from the fired gis_export import graph).

Module	Reason it fired
<code>analytics/gis/gis_export.py</code>	Manifest 'What FIRED' explicitly names <code>analytics/gis/gis_export</code> (WS150/CF/AEP GeoTIFFs + spatial_representativeness) as FIRED via the ERA5-grid retrieval path during the wind stack run.
<code>analytics/infeasibility_diagnostics.py</code>	Named in manifest 'What FIRED' orchestration list (<code>infeasibility_diagnostics</code>). Note: it only enriches an optimizer FAILURE path, but the manifest declares it fired; the lender case is a successful solve so the diagnostic detail is import-loaded/available (manifest verdict governs). (inferred nuance)
<code>analytics/irr_bridge.py</code>	Imported by <code>app/reports/report_model.irr_bridge_block</code> (<code>build_project_equity_irr_bridge_from_run</code>), which builds the report model consumed by the fired <code>capital_risk_emit</code> and <code>feasibility_sections</code> paths; the lender case publishes <code>project_irr/equity_irr</code> so the bridge is populated during the feasibility deliverable.
<code>analytics/loader/aep_loader.py</code>	The finance run reads the committed resource <code>aep_summary_path</code> (<code>scenarios/aep_summary_dutchbay_10mw.json</code>) via <code>load_aep_from_summary</code> , and the fired wind stack (<code>aep_tornado -> aep_summary_builder</code> path, APPROVED_SOURCES/provenance) imports this loader; it is the committed-summary loader for the lender case.
<code>analytics/mc/__init__.py</code>	Package imported when the capital-risk emitter (<code>emit_capital_risk_report=true, n_trials=2000</code>) called <code>analytics.mc.engine.run_monte_carlo_analysis</code> ; manifest line 19 names 'the analytics/mc/* engine' as fired.
<code>analytics/mc/aggregate.py</code>	<code>MonteCarloEngine.run()</code> calls <code>aggregate_trials</code> on every run; fired via the 2000-trial capital-risk MC (manifest line 19).
<code>analytics/mc/convergence.py</code>	<code>MonteCarloEngine.run()</code> unconditionally attaches <code>convergence_diagnostic</code> and <code>percentile_ci_diagnostic</code> to <code>result.metadata</code> (engine lines 982/987); fired via the 2000-trial capital-risk MC (manifest line 19).
<code>analytics/mc/correlation.py</code>	Imported and exercised by the MC engine on the capital-risk run: <code>init</code> calls <code>load_correlation_from_config + align_correlation_to_params</code> , and the <code>capital_risk_emit</code> docstring names 'LHS + Iman-Conover correlation' (manifest line 19); the lender scenario's correlation matrix drives <code>apply_correlation_structure</code> .
<code>analytics/mc/covenant.py</code>	<code>MonteCarloEngine.init</code> calls <code>resolve_min_dscr_covenant</code> (re-exported as <code>_resolve_min_dscr_covenant</code>) on every construction; fired via the capital-risk MC (manifest line 19).
<code>analytics/mc/degradation.py</code>	<code>MonteCarloEngine.run()</code> calls <code>apply_degradation_if_enabled</code> for every trial (engine line 863); the function executes even when degradation is disabled, so it fired on the capital-risk MC (manifest line 19).
<code>analytics/mc/engine.py</code>	The production lender MC; <code>capital_risk_emit</code> imports <code>run_monte_carlo_analysis</code> and runs it at <code>n_trials=2000</code> (manifest line 19). Sobol/FX-calibrated/degradation branches stayed off for the lender scenario but the engine itself fired.
<code>analytics/mc/samplers.py</code>	<code>generate_lhs_samples</code> fired via the default-LHS capital-risk MC (manifest line 19). <code>generate_sobol_samples</code> is opt-in (<code>monte_carlo.sampler: sobol</code>) and the lender scenario uses LHS, so the Sobol branch itself did not run.
<code>analytics/output_paths.py</code>	Named in manifest 'What FIRED' (<code>output_paths</code>); <code>run_full_pipeline_v14.py</code> and the fired report/workbook emitters resolve their default output directories through <code>resolve_output_dir</code> (<code>run_scoped=False</code>).
<code>analytics/pipeline_analytics_v14.py</code>	Named in manifest 'What FIRED' orchestration list; the lender config declares report-only detectors and the <code>pipeline_analytics_v14</code> layer is part of the fired orchestration path.
<code>analytics/pipeline_v14_enhanced.py</code>	Named in manifest 'What FIRED'; it is the entry point <code>run_full_pipeline_v14.py</code> invokes as <code>analytics.pipeline_v14_enhanced.run_v14_pipeline</code> for the canonical lender case + 8-scenario suite.
<code>analytics/portfolio/__init__.py</code>	Package imported by the fired <code>emit_tech_comparison</code> path (<code>app/reports/tech_comparison_emit -> analytics/portfolio/*</code>), which the manifest lists as FIRED with an explicit wind/hybrid/solar scenario list.
<code>analytics/portfolio/generation_aggregator.py</code>	Named explicitly in the manifest's Emitters-FIRED list (<code>emit_tech_comparison=true -> analytics/portfolio/* generation_aggregator</code>) with the <code>hybrid_windsolar / solar_only</code> scenarios in the compared-config list.
<code>analytics/portfolio/multi_tech_tornado.py</code>	Named explicitly in the manifest's Emitters-FIRED list (<code>emit_tech_comparison=true -> analytics/portfolio/* multi_tech_tornado</code>); its discovery helpers are also imported by <code>generation_aggregator/tech_wbs</code> , which fired.
<code>analytics/portfolio/tech_wbs.py</code>	Named explicitly in the manifest's Emitters-FIRED list (<code>emit_tech_comparison=true -> analytics/portfolio/* tech_wbs</code>); also lazily consumed by <code>generation_aggregator</code> for the ARCH-2 operating-margin CFADS split during the hybrid runs.

Module	Reason it fired
analytics/power_curves/oem_parser.py	Manifest states aep_tornado (tornado_from_config, FIRED) pulled analytics/power_curves/oem_parser (POWER_CURVE_STORE / CANONICAL_CURVE_KEY) when building the AEP tornado for the report.
analytics/reproducible_workbook.py	Manifest flags reproducible_workbook as 'possibly' firing under emit_executive_workbook; confirmed fired because analytics/executive_workbook.build_executive_workbook calls normalize_xlsx_reproducible on every workbook it writes.
analytics/run_manifest.py	Named in manifest 'What FIRED' (run_manifest); pipeline_v14_enhanced imports and calls build_run_manifest/engine_version so every fired pipeline output is stamped.
analytics/run_modes.py	Named in manifest 'What FIRED' (run_modes); the lender-grade feasibility run resolves its run mode through this policy table (which enforces no-toy-fallback / trial floors on the lender path).
analytics/scenario_analytics.py	Named in manifest 'What FIRED' (scenario_analytics); the fired feasibility run exercised the canonical lender case plus the 8-scenario deterministic suite, which is the batch-scenario range this orchestrator drives.
analytics/scenario_loader.py	Named in manifest 'What FIRED' (scenario_loader); load_scenario_config is called by the fired pipeline/gateway to read scenarios/dutchbay_lendercase_2025Q4.yaml and the 8-scenario suite.
analytics/schema_guard.py	Named in manifest 'What FIRED' as 'schema_guard (strict validation)'; validate_config_for_v14 is invoked by pipeline_v14_enhanced on the lender config before cashflow runs.
analytics/three_statement.py	Named in manifest 'What FIRED' orchestration list (three_statement); builds the lender-report statements from the fired pipeline's enriched annual_rows/debt_result.
analytics/wind/__init__.py	The fired module analytics.wind.aep_tornado is imported as analytics.wind.aep_tornado, so the analytics.wind package init executes at import time; per manifest guidance init is fired when its package is imported by a fired module. (Note: this init eagerly imports pipeline_aep_v14 + wind_integration, but those transitive imports are the not-fired feature modules themselves.)
analytics/wind/aep_tornado.py	Manifest: analytics/wind/aep_tornado (tornado_from_config) FIRED by hand this session; it pulled losses_model, power_curves/oem_parser, and bankable_aep density helpers to produce the report's AEP tornado.
analytics/wind/losses_model.py	Manifest: the FIRED aep_tornado 'pulled analytics/wind/losses_model' (apply_losses + default_loss_taxonomy) to apply the loss stack in the tornado net-AEP computation.
app/reports/__init__.py	Package imported when app/reports/capital_risk_emit (fired, manifest line 19) builds a ReportContext and renders the lender HTML report; the report presentation layer executed.
app/reports/capital_risk_emit.py	Manifest line 19: emit_capital_risk_report=true (n_trials=2000) fired app/reports/capital_risk_emit + analytics/capital_risk_layer_v14 + the analytics/mc engine (VaR/CVaR/breach-prob + npv_distribution PNG).
app/reports/renderer.py	Invoked by app/reports/capital_risk_emit (fired, manifest line 19) to render the lender HTML report from the assembled ReportContext (HTML/Jinja2 path).
app/reports/report_config.py	Loaded by build_report_context when the fired capital_risk_emit assembles the lender report (manifest line 19); the report presentation config was read to format KPIs and covenant thresholds.
app/reports/report_model.py	build_report_context is the gateway app/reports/capital_risk_emit calls to assemble the lender report (fired, manifest line 19); the report content model executed to produce the rendered HTML.
app/reports/tech_comparison_emit.py	Manifest line 21: emit_tech_comparison=true (with an explicit wind/hybrid/solar scenario list) fired app/reports/tech_comparison_emit + analytics/portfolio + solar_resource producers.
constants.py	Universal physical-constants module imported transitively by the finance/analytics stack that ran for the lender case; manifest treats bootstrap/config as fired via import-time (inferred).
finance/__init__.py	Package import for the finance stack, which the fired run_full_pipeline_v14 -> analytics.pipeline_v14_enhanced exercised (manifest lists the entire finance/ subsystem as fired).
finance/bess_revenue.py	The fired cashflow_v14 imports and unconditionally calls resolve_bess_specs / bess_revenue_lkr_for_year / bess_augmentation_capex_lkr_for_year (lines 157/449/537); for the wind scenarios these execute as a no-op returning None/0.0. (Manifest lists it under the BESS track, but the code path fires as a byte-identical no-op in the wind pipeline.)
finance/cashflow_v14.py	Manifest explicitly names cashflow_v14 in the finance modules fired by the lendercase/8-scenario pipeline run.
finance/cashflow_v14_contracts.py	Manifest names cashflow_v14_contracts among the fired finance modules; imported and used by the fired cashflow_v14 engine.

Module	Reason it fired
finance/cashflow_v14_fx.py	Manifest names cashflow_v14 _fx among fired finance modules; the deterministic scenario reads the FX curve to convert CFADS to USD for debt sizing/DSCR.
finance/cashflow_v14_params.py	Manifest names cashflow_v14 _params among fired finance modules; the fired cashflow_v14 engine builds/validates params through it.
finance/cashflow_v14_production.py	Manifest names cashflow_v14 _production among fired finance modules; production/revenue is the core of every scenario's CFADS.
finance/cashflow_v14_tax.py	Manifest names cashflow_v14 _tax among fired finance modules; the tax shield/CIT is computed for the lender case and all 8 scenarios.
finance/cashflow_v14_utils.py	Manifest names cashflow_v14 _utils among fired finance modules; used pervasively for config resolution inside the fired cashflow engine.
finance/debt_v14.py	Manifest explicitly names debt_v14 among the finance modules fired by the pipeline; debt sizing and DSCR are core lender-case outputs.
finance/epc_helper_v14.py	Manifest explicitly names epc_helper_v14 among the finance modules fired by the pipeline; capex breakout feeds debt sizing and cashflow.
finance/equity_distribution_v14_hydra.py	Manifest explicitly names equity_distribution_v14_hydra among the finance modules fired by the pipeline; it builds the equity waterfall from the pipeline payload.
finance/equity_v14.py	Manifest names equity_v14 among fired finance modules; equity IRR (canon eqIRR) is a headline lender-case KPI.
finance/import_lexies.py	Manifest explicitly names import_lexies among the finance modules fired by the pipeline; imported and called by the fired cashflow_v14 and debt_v14 (returns zero uplift when no taxes_indirect block).
finance/irr.py	Manifest explicitly names irr among the finance modules fired by the pipeline; every projIRR/eqIRR/NPV KPI routes through this singleton.
finance/irr_config.py	Manifest explicitly names irr_config among the finance modules fired by the pipeline; supplies project-specific IRR search bounds to finance.irr.
finance/tech_types.py	Manifest explicitly names tech_types among the finance modules fired by the pipeline; classifies each technology block (wind + hybrid/solar scenarios) for revenue aggregation.
finance/utils.py	Manifest explicitly names utils among the finance modules fired by the pipeline; imported by the fired debt_v14, epc_helper_v14 and import_lexies.
finance/wacc_v14.py	Manifest explicitly names wacc_v14 among the finance modules fired by the pipeline; WACC is the discount rate feeding NPV and the equity/LCOS views.
run_full_pipeline_v14.py	Manifest 'What FIRED': invoked as run_full_pipeline_v14.py config=scenarios/dutchbay_lendercase_2025Q4.yaml for the canonical lender case + 8-scenario suite driving analytics.pipeline_v14_enhanced.run_v14_pipeline.
solar_resource/__init__.py	Package imported at pipeline module-load: the fired run_full_pipeline_v14 entrypoint imports solar_resource.cashflow_adapter (line 146), so solar_resource/init (and its eager submodule imports) execute. Per the manifest's init rule (fired if the package is imported by a fired module).
wind_resource/__init__.py	Manifest names era5_retrieval as FIRED for the wind stack; importing wind_resource.* (era5_retrieval imports EnergyCalculator via this package) executes this init at import time. (inferred)
wind_resource/bankable_aep.py	Manifest: the aep_tornado FIRED and 'pulled ... density helpers from wind_resource/bankable_aep'; density/gross helpers were touched via the tornado (a full fresh bankable_aep run did NOT occur, but the module fired).
wind_resource/energy_calculator.py	Manifest: era5_retrieval.compute_site_aep FIRED, and compute_site_aep drives net AEP through EnergyCalculator (this is the timeseries-integration AEP path era5_retrieval invokes). (inferred)
wind_resource/era5_retrieval.py	Manifest names it FIRED explicitly (ARCO single-point ERA5 2005-2024, incl. retrieve_era5_timeseries, build_hub_height_series, validate_coverage, compute_site_aep, build_production_wind_rose — the report's wind rose came from here — and it called long_term_trend).
wind_resource/layout_optimizer.py	Manifest explicitly: 'wind_resource/layout_optimizer (TopFarm/PyWake micro-siting) — FIRED (+1.36% candidate)'.
wind_resource/long_term_trend.py	Manifest: era5_retrieval 'called wind_resource/long_term_trend (Mann-Kendall/Sen). FIRED.' (also reiterated 'long_term_trend (FIRED — see above)').

4b. Not included — full list with reasons

Module	Cov	Reason not included
<code>analytics/aep_provenance.py</code>	•	Manifest explicitly lists <code>analytics/aep_provenance</code> among the 'wind modules NOT fired this pass' (grouped with <code>aep_reconciliation</code>); finance used the committed <code>aep_summary_dutchbay_10mw.json</code> rather than re-loading an authored scenario declaring <code>resource.power_curve.source_id</code> , so this load-time provenance guard was not triggered.
<code>analytics/aep_reconciliation.py</code>	•	Manifest explicitly names the top-level <code>analytics/aep_reconciliation</code> among the wind modules NOT fired this pass (it runs at authored-scenario load, not on the committed <code>aep_summary</code> the finance pipeline read); no fresh authored-scenario load exercised the reconciliation guard.
<code>analytics/ capital_structure_optimizer_v14.py</code>	•	Manifest lists <code>capital_structure_optimizer_v14</code> explicitly under 'standalone sensitivity/optimization NOT run as separate steps' — it is opt-in and only invoked via its dedicated <code>analytics/cli/cli_capital_structure_optimize_hydra.py</code> CLI, which the lender feasibility run did not call (risk was covered by the 2000-trial capital-risk MC + deterministic 8-scenario range).
<code>analytics/casper/__init__.py</code>	•	Manifest flags the CASPER MC facet as fired ONLY if the capital-risk emitter routes through it; verified <code>capital_risk_emit.py</code> / <code>capital_risk_layer_v14.py</code> do NOT import <code>analytics.casper</code> (only <code>contracts_v14</code> comments, <code>sensitivity/tail_risk</code> (not fired), and <code>mc/covenant</code> reference it), so the package was not imported during the study run.
<code>analytics/casper/casper_payload.py</code>	•	Manifest gates CASPER on <code>capital_risk_emit</code> importing <code>casper</code> ; it does not (capital-risk MC routes through <code>analytics.mc.engine</code> + <code>capital_risk_layer_v14</code> , neither of which imports <code>casper</code>), so <code>build_casper_payload</code> was not exercised by the wind feasibility deliverable.
<code>analytics/casper/casper_v14.py</code>	•	The feasibility run used <code>run_full_pipeline_v14</code> + the capital-risk emitter, not the CASPER orchestrator entry point; no fired module imports <code>analytics.casper</code> , so this orchestrator did not run (would need an explicit <code>evaluate_with_casper_tail_risk_and_payload</code> call).
<code>analytics/cli/__init__.py</code>	•	The feasibility run used <code>run_full_pipeline_v14.py</code> (<code>pipeline_v14_enhanced</code>), not the standalone <code>analytics</code> CLIs; the manifest lists standalone MC/sensitivity/optimizer runs as NOT run as separate steps, so this CLI package was not imported this pass. (inferred)
<code>analytics/cli/ cli_capital_structure_optimize_hydra.py</code>	•	Manifest lists <code>capital_structure_optimizer_v14/optimization_v14</code> among the standalone steps NOT run this pass; this CLI is invoked explicitly and never runs inside the pipeline, and the feasibility deliverable did not invoke it.
<code>analytics/cli/cli_monte_carlo_hydra.py</code>	•	Manifest: the full-population MC (<code>analytics/monte_carlo_v14</code> , 100k) was NOT run as a separate step (risk covered by the 2000-trial capital-risk emitter instead); the standalone MC CLI endpoint was not invoked.
<code>analytics/cli/cli_sensitivity_hydra.py</code>	•	Manifest: standalone sensitivity (<code>analytics/sensitivity_v14</code> and this CLI path) was NOT run as a separate step this pass (deterministic 8-scenario range + AEP tornado covered sensitivity); the CLI endpoint was not invoked.
<code>analytics/contracts/__init__.py</code>	•	Manifest names <code>contracts_v14</code> as fired, but the fired modules import from <code>analytics.contracts_v14</code> directly; this facade package is only exercised if a caller imports via <code>analytics.contracts</code> , which the feasibility path does not do. (inferred)
<code>analytics/core/parameter_solvers.py</code>	•	Docstring states these are read-only analyst tools with NO committed-scenario caller and no Hydra CLI wiring (#615 deferred); the feasibility pipeline run does not invoke any solver, so they did not fire.
<code>analytics/core/sensitivity_runner.py</code>	•	Manifest: standalone sensitivity was NOT run as a separate step this pass (covered by the deterministic 8-scenario range + AEP tornado); this runner is only invoked by the sensitivity CLIs / FastAPI /run-tornado / streamlit app, none of which fired. (inferred)
<code>analytics/cost/benchmark.py</code>	•	Its functions did not run on the wind study: <code>lcos_benchmark</code> is invoked only inside <code>scenario_analytics</code> <code>compute_lcos_suite</code> (BESS/storage LCOS, empty for a pure-wind scenario), and <code>capex_benchmark</code> is imported only by the <code>not_applicable</code> web layer <code>api/pipeline_api.py</code> ; module import alone (via <code>scenario_analytics</code>) did not exercise the benchmark logic (inferred).
<code>analytics/cost/cost_basis.py</code>	•	Its only real importer is the <code>not_applicable</code> web layer <code>api/pipeline_api.py</code> (<code>resolve_cost_basis_year</code>); the fired deterministic pipeline references it only in a <code>losses_model</code> docstring, so the cost-basis resolver did not run in the study.

Module	Cov	Reason not included
analytics/cost/estimate_class.py	•	Imported only by api/pipeline_api.py (not_applicable web layer) and by analytics.cost.mc_capex; the deterministic study path does not import it and the CAPEX MC (mc_capex) did not fire, so resolve_accuracy_band was not exercised.
analytics/cost/mc_capex.py	•	No fired module imports run_capex_mc (only debt_v14/tech_wbs docstring mentions); the manifest lists the full-population/standalone MC & sensitivity steps as not run (risk was covered by the 2000-trial capital-risk MC + 8-scenario range + AEP tornado), so this dedicated CAPEX-MC step did not fire.
analytics/dscr_sensitivity.py	•	Manifest names dscr_sensitivity in the 'standalone sensitivity NOT run as separate steps' list; it is a separate Hydra step not invoked by run_full_pipeline_v14 for the lender case, and DSCR-range visibility came from the 8-scenario suite rather than this dual-DSCR sweep.
analytics/fx/fx_calibration.py	•	Opt-in: analytics.mc.engine only initializes it when a distribution: fx_calibrated MC param is declared; the fired capital-risk MC uses the lender scenario's plain-uniform fx driver (parameter list [capacity_factor, tariff, opex, capex, fx, curtailment], no fx_calibrated), so calibrate_from_config was never reached, and fx_sensitivity_real / full-population MC did not fire.
analytics/fx/fx_history.py	•	Feeds only fx_calibration (the opt-in fx_calibrated MC driver), which the lender scenario does not declare; also depends on the pinned inputs/fxdata/ history artifact. Since the calibrated-FX MC path did not fire, load_pinned_history was not reached (inferred).
analytics/fx_sensitivity_real.py	•	Manifest explicitly lists fx_sensitivity_real among the standalone sensitivity steps NOT run this pass; the FX curve/spread was read only deterministically by the 8 scenarios, and no separate FX-shock/hedge/spread sweep was executed.
analytics/gis/boundary_clip.py	•	Manifest 'GIS layers NOT fired' explicitly lists analytics/gis/boundary_clip; the ERA5-grid-only export produced full-grid rasters and no project-boundary polygon clip was requested this pass.
analytics/gis/dem_ingest.py	•	Manifest 'GIS layers NOT fired' names analytics/gis/dem_ingest; the terrain layers need a GWA/DEM raster (not cached) and the manifest notes the elevation DEM dependency is missing, so it did not run.
analytics/gis/exclusion_mask.py	•	Manifest 'GIS layers NOT fired' explicitly lists analytics/gis/exclusion_mask; it consumes the #829 DEM slope layer (which did not run — DEM dep missing) plus setback/protected-area geometry that the ERA5-only export pass did not supply.
analytics/gis/gwa_ingest.py	•	Manifest 'GIS layers NOT fired' explicitly lists analytics/gis/gwa_ingest; the export ran only the ERA5-grid path and needs a local GWA 250 m raster (not cached) that was not supplied this pass.
analytics/gis/landcover_roughness.py	•	Manifest 'GIS layers NOT fired' explicitly lists analytics/gis/landcover_roughness; the GIS export ran only the ERA5-grid path and this land-suitability layer needs a landcover raster not cached this pass.
analytics/gis/mcdm_suitability.py	•	Manifest 'GIS layers NOT fired' explicitly lists analytics/gis/mcdm_suitability; it composes the upstream GWA/DEM/roughness/exclusion criterion rasters, none of which ran this pass (terrain/landcover rasters not cached), so the suitability surface did not fire.
analytics/gis/rix.py	•	Manifest 'GIS layers NOT fired' explicitly lists analytics/gis/rix; it consumes the #829 DEM slope layer, which did not run this pass (DEM/elevation dependency missing).
analytics/grid/__init__.py	•	Manifest names the ENTIRE analytics/grid/* capability as advisory default-off; the lender scenario sets grid.study_enabled=false (master gate), so this package's screens were never invoked during the wind feasibility deliverable.
analytics/grid/capabilities/__init__.py	•	Manifest lists analytics/grid/capabilities/* under the advisory default-off grid capability; gated off by grid.study_enabled=false, so no capability plug-in dispatch ran during the feasibility deliverable (inferred).
analytics/grid/capabilities/solar.py	•	Part of the advisory default-off analytics/grid/capabilities/* subsystem; gated by grid.study_enabled=false, so solar grid-capability screening did not fire in this wind study (inferred).
analytics/grid/capabilities/wind.py	•	Part of the advisory default-off analytics/grid/capabilities/* subsystem; wind capability screening is gated by grid.study_enabled=false, distinct from the wind AEP stack that did fire, so it did not run (inferred).

Module	Cov	Reason not included
<code>analytics/grid/curtailment_qsts.py</code>	•	Manifest lists curtailment_qsts under the advisory default-off grid capability; gated off by <code>grid.study_enabled=false</code> and needs a real feeder model plus the OpenDSSDirect [grid] extra, so it did not fire.
<code>analytics/grid/evaluate_grid.py</code>	•	Manifest lists evaluate_grid under the advisory default-off grid capability; the study is gated by <code>grid.study_enabled=false</code> so the gateway short-circuits and did no work during the feasibility run.
<code>analytics/grid/frequency_response.py</code>	•	Manifest lists frequency_response under the advisory default-off grid capability; gated off by <code>grid.study_enabled=false</code> , so the de-load sizer did not run in the wind study.
<code>analytics/grid/grid_interface_schema.py</code>	•	Manifest lists grid_interface_schema under the advisory default-off grid capability; validate-when-present only runs if a scenario declares a top-level grid block, and the lender scenario has <code>grid.study_enabled=false</code> , so it did not fire (inferred).
<code>analytics/grid/harmonics.py</code>	•	Manifest lists harmonics under the advisory default-off grid capability; gated off by <code>grid.study_enabled=false</code> , so the power-quality screen did not run this pass.
<code>analytics/grid/hybrid/__init__.py</code>	•	Manifest lists analytics/grid/hybrid/* under the advisory default-off grid capability; gated off by <code>grid.study_enabled=false</code> , so hybrid aggregation did not run this pass (inferred).
<code>analytics/grid/hybrid/frequency_response.py</code>	•	Manifest lists analytics/grid/hybrid/* under the advisory default-off grid capability; gated by <code>grid.study_enabled=false</code> , so the combined frequency-response study did not run (inferred).
<code>analytics/grid/hybrid/poc_aggregation.py</code>	•	Manifest lists analytics/grid/hybrid/* under the advisory default-off grid capability; gated by <code>grid.study_enabled=false</code> , so the hybrid aggregation did not fire.
<code>analytics/grid/reactive_screen.py</code>	•	Manifest lists reactive_screen under the advisory default-off grid capability; gated off by <code>grid.study_enabled=false</code> and requires the pandapower [grid] extra, so it did not fire this pass.
<code>analytics/grid/ride_through.py</code>	•	Manifest lists ride_through under the advisory default-off grid capability; gated off by <code>grid.study_enabled=false</code> and its ANDES solve needs the [grid] extra, so it did not fire in the wind study.
<code>analytics/grid/ride_through_poc.py</code>	•	Manifest names ride_through_poc explicitly under the advisory default-off grid capability; a deprecated shim over the ride_through core, gated off by <code>grid.study_enabled=false</code> , so it did not run.
<code>analytics/grid/short_circuit.py</code>	•	Manifest lists short_circuit explicitly under the advisory default-off grid capability; gated off by <code>grid.study_enabled=false</code> and needs a real utility fault level, so it did not run this pass.
<code>analytics/mc/exports.py</code>	•	Its only consumer is analytics.casper.casper_payload (build_casper_risk_blocks). The fired capital-risk emitter (capital_risk_emit / capital_risk_layer_v14) does NOT import casper, and the CASPER mc_risk facet did not run this pass (manifest line 36), so these export builders were not invoked.
<code>analytics/monte_carlo_v14.py</code>	•	Manifest explicitly lists analytics/monte_carlo_v14 (the 100k-scenario full-population MC) as NOT run: its overlay dutchbay_mc_enhanced_2025Q4.yaml is missing base fields and cannot run standalone; risk was covered by the 2000-trial capital-risk MC (analytics/mc engine), the 8-scenario range, and the AEP tornado — not this standalone path.
<code>analytics/optimization_v14.py</code>	•	Manifest explicitly lists optimization_v14 among the standalone optimization steps NOT run this pass; the feasibility run evaluated the fixed lender config and 8 scenarios rather than searching a parameter for a KPI-optimal point.
<code>analytics/portfolio/poi_curtailment.py</code>	•	Opt-in: resolve_shared_poi_curtailment returns None unless a scenario declares generation.shared_poi.limit_mw AND >=2 techs supply generation.technologies.<tech>.hourly_profile_mw (hourly TMY/wind profiles tracked in #529, not present). The manifest also states DutchBay's 220kV line is a separate CEB project so the POI is not binding, and the emitter/portfolio fired list does not name it. (inferred)
<code>analytics/sensitivity/__init__.py</code>	•	Standalone sensitivity was not run as a separate feasibility step; the AEP tornado used analytics.wind.aep_tornado, not this suite. Manifest line 33 lists analytics/sensitivity_v14 and analytics/sensitivity/* (global_sa, tail_risk) among the not-run standalone steps; no fired module imports this package.

Module	Cov	Reason not included
<code>analytics/sensitivity/adapters.py</code>	•	Only imported by the sensitivity engine and interaction modules, none of which ran this pass (standalone sensitivity + interaction grid not fired; manifest lines 31, 33).
<code>analytics/sensitivity/docstrings.py</code>	•	Pure documentation constants belonging to the sensitivity suite, which did not run standalone this pass (manifest line 33); no fired module imports them.
<code>analytics/sensitivity/dscr.py</code>	•	Wraps <code>build_one_way_sensitivity_suite</code> ; the <code>dscr_sensitivity</code> standalone step is explicitly among the not-run items (manifest line 33) and standalone sensitivity was not part of this feasibility pass.
<code>analytics/sensitivity/engine.py</code>	•	Standalone deterministic sensitivity (<code>analytics/sensitivity_v14</code> and this suite) was not run as a separate step (manifest line 33); note the MC engine imports only its <code>_resolves_in_config</code> helper for dead-key detection, but the tornado orchestration endpoints did not fire.
<code>analytics/sensitivity/export.py</code>	•	Consumes <code>SensitivitySuite</code> output from the sensitivity engine, which did not run standalone this pass (manifest line 33).
<code>analytics/sensitivity/global_sa.py</code>	•	Explicitly named as not run this pass: manifest line 33 lists <code>analytics/sensitivity/global_sa</code> (Sobol/PAWN) among the standalone steps not executed. Also gated by the optional SALib dependency (<code>_require_salib</code>).
<code>analytics/sensitivity/interaction.py</code>	•	The <code>emit_interaction_grid</code> emitter is gated off — the lender scenario declares no <code>interaction_grid</code> metric/param block and the emitter fail-louds — so <code>analytics/sensitivity/interaction</code> did NOT run (manifest line 31).
<code>analytics/sensitivity/optimizer.py</code>	•	The <code>sensitivity_pareto</code> / optimization standalone steps were not run this pass (manifest line 33); the <code>pymoo</code> backend is additionally an optional call-time dependency.
<code>analytics/sensitivity/tail_risk.py</code>	•	Manifest line 33 names <code>analytics/sensitivity/tail_risk</code> (CVaR standalone) among the not-run steps; the tornado enricher is reached only via the sensitivity engine (<code>enrich_tail_risk</code> opt-in), which did not fire, and the distributional path is explicitly unwired.
<code>analytics/sensitivity/tax.py</code>	•	Thin wrapper over <code>build_one_way_sensitivity_suite</code> ; standalone deterministic sensitivity was not part of this feasibility pass (manifest line 33).
<code>analytics/sensitivity/validation.py</code>	•	A CI/test QA checker for sensitivity parameter sweeps (imports <code>evaluate_with_overrides</code> from <code>analytics.evaluate_scenario</code>); not part of the feasibility deliverable run, and standalone sensitivity was not fired (manifest line 33) (inferred).
<code>analytics/sensitivity_pareto.py</code>	•	Manifest explicitly lists <code>sensitivity_pareto</code> among the standalone sensitivity/optimization steps NOT run this pass; no Pareto multi-objective search was invoked for the deterministic feasibility deliverable.
<code>analytics/sensitivity_v14.py</code>	•	Manifest lists <code>analytics/sensitivity_v14</code> among the standalone sensitivity steps NOT run this pass; the deterministic 8-scenario range plus the AEP tornado covered sensitivity instead of the standalone one-way suite.
<code>analytics/simulation/monte_carlo_aep.py</code>	•	Manifest lists the 100k-scenario AEP MC / <code>mc_aep_weibull</code> as NOT fired; its only importers (<code>analytics/wind/pipeline_aep_v14</code> , <code>analytics/wind/wind_integration</code>) are in the wind-modules-not-fired list — the finance run read the committed <code>aep_summary</code> rather than re-running the AEP MC.
<code>analytics/wacc_sensitivity.py</code>	•	Manifest explicitly lists <code>wacc_sensitivity</code> among the standalone sensitivity analyses NOT run as separate steps this pass; it is a separate ke-band sweep not wired into the fired 8-scenario deterministic pipeline.
<code>analytics/wind/aep_summary_builder.py</code>	•	Manifest lists <code>aep_summary_builder</code> under 'Wind modules NOT fired this pass'; the finance run consumed the already-committed <code>aep_summary_dutchbay_10mw.json</code> (<code>resource.aep_summary_path</code>) rather than rebuilding the summary from config, so this builder did not run.
<code>analytics/wind/crossval_interface_schema.py</code>	•	The <code>resource.crossval</code> block is opt-in/default-off and the lender scenario does not declare it; <code>schema_guard</code> only imports+registers the 'crossval' module when it is in the validate list, and the finance run validates the default ['cashflow', 'debt'] only. Manifest also lists <code>wind_resource/crossval</code> as NOT fired.
<code>analytics/wind/era5_interface_schema.py</code>	•	<code>schema_guard</code> imports+registers the 'era5' module only when 'era5' is in the validate list, but the finance run validates the default ['cashflow', 'debt']; the ERA5 retrieval that DID fire went through <code>wind_resource.era5_retrieval/ERA5RequestConfig</code> directly, not this schema-guard path. Not named in the manifest's fired list (inferred).

Module	Cov	Reason not included
analytics/wind/mc_aep_weibull.py	•	Manifest explicitly lists mc_aep_weibull under 'Wind modules NOT fired this pass'; resource risk in the deliverable was covered by the 2000-trial capital-risk MC + the deterministic 8-scenario range + the AEP tornado, not this Weibull-MC AEP path.
analytics/wind/pipeline_aep_v14.py	•	Manifest lists pipeline_aep_v14 under 'Wind modules NOT fired this pass'; the finance pipeline read the committed aep_summary directly (resource.aep_summary_path) and did not route the wind->AEP chain through this integrator, and monte_carlo_aep (its optional dep) was likewise not run.
analytics/wind/siting_metadata.py	•	Manifest explicitly lists siting_metadata under 'Wind modules NOT fired this pass'; it is only consumed when building an AEP summary from config (aep_summary_builder), which did not run since the finance case used the committed summary.
analytics/wind/wind_integration.py	•	Manifest lists wind_integration under 'Wind modules NOT fired this pass'; the lender finance run consumed the committed aep_summary directly and did not use this bridge to inject AEP into config or run the AEP MC (its module namespace is imported by the wind package init , but its functions were not exercised).
analytics/wind/wind_interface_schema.py	•	Manifest lists wind_interface_schema under 'Wind modules NOT fired this pass'; schema_guard imports+registers the 'wind' module only when 'wind' is in the validate list, but the strict finance run validates the default ['cashflow', 'debt'] set, so this schema was not registered/enforced this pass.
analytics/wind/wind_rose.py	•	Manifest explicitly states the report's wind rose came from wind_resource.era5_retrieval.build_production_wind_rose, NOT this standalone analytics/wind/wind_rose.py module, which did not fire this pass.
app/reports/grid_screening_emit.py	•	Manifest line 32: the entire grid capability including app/reports/grid_screening_emit did NOT run — scenario grid.study_enabled:false (master default-off gate) and emit_grid_screen:false; KPI-neutral by design.
app/reports/interaction_grid_emit.py	•	Manifest line 31: emit_interaction_grid → app/reports/interaction_grid_emit + analytics/sensitivity/interaction did NOT run — the lender scenario declares no interaction_grid metric/param block and the emitter fail-louds.
app/reports/wind_rose_plot.py	•	Only fires when the report context carries a wind_rose block from analytics/wind/wind_rose.py, which manifest line 34 states did NOT fire (the report's rose came from era5_retrieval.build_production_wind_rose, run by hand and outside the capital_risk_emit report path). (inferred)
finance/cashflow/__init__.py	•	Empty aspirational package: the fired pipeline imports the flat finance.cashflow_v14* modules directly, not this finance.cashflow package (no consumer imports it), so this init did not execute this pass (inferred).
finance/equity/__init__.py	•	Convenience re-export package; the fired pipeline imports finance.equity_v14 and finance.equity_distribution_v14_hydra directly, so this finance.equity namespace package was not necessarily imported this pass (inferred).
finance/grid/__init__.py	•	Manifest lists finance/grid/* under the advisory default-off grid capability; consulted only by the grid study which is gated off by grid.study_enabled=false, so it did not fire in the wind feasibility run (inferred).
finance/grid/grid_types.py	•	Manifest lists finance/grid/* under the advisory default-off grid capability; these classifiers are consumed only by the grid study, gated by grid.study_enabled=false, so they were not exercised (inferred).
finance/self_curtailment_v14.py	•	Manifest states finance/self_curtailment_v14 did NOT run: the grid capability is default-off (scenario grid.study_enabled: false and no grid.qsts.finance_wiring.enabled opt-in), so the resolver short-circuits to 0.0 before any QSTS import.
run_scenario_analytics_v14.py	•	The feasibility deliverable used the canonical run_full_pipeline_v14.py path (manifest 'What FIRED'); this deliberately-lighter batch-compare CLI (PIPE-1) was not the run entrypoint and did not fire this pass (inferred).
scripts/export_to_excel.py	•	The Excel deliverable was produced by the wired emit_executive_workbook emitter (analytics/executive_workbook) per manifest 'Emitters FIRED'; this older standalone export script was not the path used this pass (inferred).
scripts/run_fx_calibration.py	•	FX curve was read deterministically by the fired scenario (manifest: analytics/fx/* fired to the extent the scenario reads it), but this standalone calibration/inspection CLI needs network/refresh and did not run as a separate step this pass (inferred).

Module	Cov	Reason not included
<code>scripts/run_fx_sensitivity.py</code>	•	Manifest 'did NOT fire': fx_sensitivity_real and standalone sensitivity steps were not run separately (risk covered by the 2000-trial capital-risk MC + 8-scenario range + AEP tornado); this CLI wraps that un-fired path.
<code>scripts/run_global_sensitivity.py</code>	•	Manifest 'did NOT fire': analytics/sensitivity/global_sa (Sobol/PAWN) was not run as a separate step; this CLI wraps that un-fired global-SA path.
<code>scripts/run_multi_tech_tornado.py</code>	•	The multi_tech_tornado LIBRARY fired via emit_tech_comparison (manifest), but this standalone CLI wrapper was not the invocation path for the deliverable and did not run as its own step (inferred).
<code>scripts/run_tornado_from_cli.py</code>	•	Standalone one-way tornado CLI; the finance sensitivity_v14 standalone path did not fire this pass (manifest 'did NOT fire' list) and this wrapper was not the deliverable's invocation (inferred).
<code>scripts/run_wind_analysis_v14.py</code>	•	Manifest not_applicable section: attempted this pass but hit the gridded CDS failure and was replaced by the era5_retrieval module; explicitly 'mark available_not_fired'.
<code>solar_resource/bifacial_guard.py</code>	•	Module imported via the solar_resource package (cashflow_adapter imports it), but assert_monofacial_financed_cf only executes inside solar_export_to_scenario_patch, which the W4 (#614) solar-ingestion path is off for (no solar_export_path; committed hybrid/solar scenarios carry a pre-baked frozen CF 0.1685). No bifacial marker is present. (inferred)
<code>solar_resource/cashflow_adapter.py</code>	•	Imported at module-load by the fired run_full_pipeline_v14 (line 146), but its functional bridge did not execute: the optional W4 (#614) solar-resource ingestion (_apply_solar_to_scenario -> solar_export_to_scenario_patch) is OFF by default and the committed hybrid/solar scenarios declare no solar_export_path (they carry the frozen CF directly). (inferred)
<code>solar_resource/exceedance.py</code>	•	Imported with the solar_resource package but its exceedance build-up only runs via compute_solar_aep(emit_exceedance=True) or a non-P50 build_solar_cashflow_export — the offline freeze step, which did not run this pass (finance consumed the committed frozen P50 CF; pvlib producer not invoked). (inferred)
<code>solar_resource/long_term_trend.py</code>	•	The solar analogue of wind_resource.long_term_trend (which the manifest lists as fired for wind). It runs only when a solar producer freezes a SARA-3 annual-GHI table offline and calls build_solar_resource_trend_export_block; no fresh solar producer/freeze ran this pass (frozen solar CF consumed, no GHI series supplied). (inferred)
<code>solar_resource/loss_model.py</code>	•	Consumed only by compute_solar_aep when a scenario declares resource.solar.losses; the pvlib producer did not run this pass (finance used the committed frozen solar CF, no itemised loss stack recomputation). (inferred)
<code>solar_resource/pv_producer.py</code>	•	The compute path needs the optional pvlib [solar] extra and runs only as the OFFLINE freeze step; the manifest states finance consumes a frozen solar CF (committed scenarios carry 0.1685 directly, pvlib-free), so compute_solar_aep did not execute for the deliverable. (inferred)
<code>solar_resource/soiling_profile.py</code>	•	Opt-in (resource.solar.soiling_profile); consumed only inside compute_solar_aep, which did not run this pass. The committed hybrid declares no soiling_profile (soiling_profile_from_config returns None -> flat-soiling byte-identical). (inferred)
<code>solar_resource/source_quality.py</code>	•	solar_source_quality_from_config is invoked only by the OVERWRITE bridge solar_export_to_scenario_patch (to stamp provenance), which the off-by-default W4 solar-ingestion path did not run this pass (committed scenarios carry pre-baked CF, no solar_export_path). Pure metadata, KPI-neutral. (inferred)
<code>wind_resource/arco_assessment.py</code>	•	Manifest lists era5_retrieval (incl. compute_site_aep/build_production_wind_rose) as the ARCO path that FIRED by hand; this VALIDATE-mode drift wrapper around it is not named as run, and finance used the committed aep_summary rather than a fresh ARCO re-baseline. (inferred)
<code>wind_resource/cashflow_adapter.py</code>	•	Bridges the WindPipeline export into a scenario; finance read the COMMITTED scenarios/aep_summary_dutchbay_10mw.json (manifest: run_full_pipeline does NOT re-run the wind stack), so no WindPipeline export was patched in this pass. (inferred)
<code>wind_resource/config/__init__.py</code>	•	The fired ARCO path (era5_retrieval) is scenario/Hydra-config driven and does not import this config subpackage; the WindPipeline/EnergyCalculator YAML-

Module	Cov	Reason not included
		config path that would import it did not run (finance used the committed aep_summary). (inferred)
wind_resource/crossval.py	•	Opt-in via resource.crossval.enabled; the lender scenario declares no crossval block (crossval_settings returns None), and the manifest does not list any second-source cross-validation among what fired. (inferred)
wind_resource/era5_fetcher.py	•	Manifest explicitly: 'era5_fetcher (gridded — hit the CDS cost limit and was abandoned for ARCO)' — did NOT fire; the ARCO single-point era5_retrieval path was used instead.
wind_resource/era5_grid.py	•	Manifest explicitly: 'era5_grid (in wind_resource — distinct from analytics/gis/era5_grid)' among the wind modules NOT fired this pass; the GIS export used the analytics/gis/era5_grid retrieval path instead.
wind_resource/mcp.py	•	Manifest lists 'mcp' among the wind modules NOT fired this pass; it is opt-in (needs resource.wind.mcp + a mast data file), and no mast was wired — ERA5 raw resource was used.
wind_resource/power_curve_sourcing.py	•	Manifest lists 'power_curve_sourcing' among the wind modules NOT fired; it is a curve-ingest tool (the committed power curve was already in the store / POWER_CURVE_STORE consumed via oem_parser), not exercised by the study run.
wind_resource/weibull_fit.py	•	Manifest explicitly: 'weibull_fit (fit was already committed in the scenario)' among wind modules NOT fired; its only importers (arco_assessment, crossval, mcp) also did not run this pass.
wind_resource/wind_analyzer.py	•	Manifest lists 'wind_analyzer' among the wind modules NOT fired; it belongs to the WindPipeline/CSV diagnostic path, which did not run (finance used the committed aep_summary; ARCO path fired era5_retrieval instead).
wind_resource/wind_pipeline.py	•	Manifest lists 'wind_pipeline' among the wind modules NOT fired; it drives ERA5Fetcher (the abandoned gridded path) and the WindPipeline export→cashflow_adapter route, neither of which ran (ARCO era5_retrieval fired instead).
analysis_tools/___init___py	○	Manifest lists analysis_tools/* under 'Legacy / sandbox / examples' not part of a wind feasibility deliverable; the package doc itself says 'NOT for production pipelines - use analytics/ instead', so it was never imported by the feasibility run (no pipeline/report caller).
analytics/cli/cli_sensitivity.py	○	Deprecated argparse shim (removal planned Sprint 18) superseded by cli_sensitivity_hydra.py; not part of the wind-feasibility run, which used run_full_pipeline_v14.py. (inferred)
analytics/dashboard/streamlit_app.py	○	Interactive Streamlit UI run via streamlit run; a presentation/app layer, not part of a batch wind-feasibility study run (analogous to the web-service app layer the manifest marks not_applicable). (inferred)
analytics/grid/capabilities/bess.py	○	Manifest lists analytics/grid/capabilities/bess* explicitly under not_applicable (separate BESS track, not this wind study); also under the default-off grid gate.
analytics/grid/capabilities/bess_soc.py	○	Manifest lists analytics/grid/capabilities/bess* explicitly under not_applicable (separate BESS track, not this wind study); a BESS energy-accounting foundation, also under the default-off grid gate.
analytics/pysam_sandbox/___init___py	○	Manifest lists analytics/pysam_sandbox/* under 'Legacy / sandbox / examples'; the module docstring states it is NOT imported by the core pipeline unless generation.engine=pysam, which the lender/AEP feasibility scenarios do not set (the wind stack used era5_retrieval/bankable_aep, not PySAM).
analytics/pysam_sandbox/pysam_runner.py	○	Same PySAM sandbox as its init ; requires the optional NREL-PySAM dependency and generation.engine=pysam, neither of which the feasibility run used (manifest 'Legacy / sandbox / examples'; wind AEP came from era5_retrieval/bankable_aep, not PySAM).
analytics/sensitivity/dashboard_demo.py	○	Demo/example snippet importing symbols the package does not even export (SensitivityRequest, plot_tornado_chart); executes on import failure, is not part of any feasibility path (manifest not_applicable: examples/sandbox, line 43).
api/___init___py	○	Manifest line 40 lists top-level api/ among the FastAPI web-service productization (#788) that is not exercised by a study run.
api/path_safety.py	○	Serves only the FastAPI HTTP endpoints (api.pipeline_api / api.sensitivity_api); manifest line 40 marks all of top-level api/ not part of a study run.
api/pipeline_api.py	○	

Module	Cov	Reason not included
		Web-service adapter under top-level <code>api/</code> ; manifest line 40 states the FastAPI backend was not exercised by the study run (the study used the CLI <code>run_full_pipeline_v14.py</code> , not this router).
<code>api/sensitivity_api.py</code>	○	Web-service HTTP endpoint under top-level <code>api/</code> ; manifest line 40 marks the FastAPI surface not exercised by the study run.
<code>app/__init__.py</code>	○	Root of the web-service productization package; manifest line 40 lists all of <code>app/api</code> , <code>app/jobs</code> , <code>app/services</code> , <code>app/web</code> , <code>app/models</code> as not exercised by a study run. (Note: <code>app/reports</code> fired separately.)
<code>app/api/__init__.py</code>	○	Manifest line 40 lists all of <code>app/api</code> among the FastAPI web-service (#788) not exercised by a study run.
<code>app/api/auth.py</code>	○	Authentication gate for the FastAPI web surface (<code>app/api</code>); manifest line 40 marks the web service not exercised by a study run.
<code>app/api/config.py</code>	○	Operational config for the FastAPI sync web routes; manifest line 40 marks the web service (<code>app/api</code>) not exercised by a study run.
<code>app/api/jobs_router.py</code>	○	Async job HTTP surface of the web service (<code>app/api</code> + <code>app/jobs</code>); manifest line 40 marks it not exercised by a study run.
<code>app/api/main.py</code>	○	The FastAPI application endpoint of the web service (#788); manifest line 40 states it was not exercised by the study run (the study ran the CLI pipeline directly).
<code>app/api/responses.py</code>	○	Response contract for the FastAPI web boundary (<code>app/api</code>); manifest line 40 marks the web service not exercised by a study run.
<code>app/api/surface.py</code>	○	Wizard result-surface contract for the FastAPI web boundary; manifest line 40 marks <code>app/api</code> not exercised by a study run.
<code>app/jobs/__init__.py</code>	○	Manifest line 40 lists all of <code>app/jobs</code> among the web service (#788) not exercised by a study run.
<code>app/jobs/config.py</code>	○	Operational config for the async job path (<code>app/jobs</code>); manifest line 40 marks the web service not exercised by a study run.
<code>app/jobs/models.py</code>	○	Domain models for the async web-service job path (<code>app/jobs</code>); manifest line 40 marks it not exercised by a study run.
<code>app/jobs/redis_store.py</code>	○	Redis persistence for the async web-service job path (optional [<code>jobs</code>] extra); manifest line 40 marks the web service not exercised by a study run.
<code>app/jobs/runner.py</code>	○	Orchestrator for the async web-service job path (<code>app/jobs</code>); manifest line 40 marks the web service not exercised by a study run (and its live ERA5 <code>default_assessment</code> needs Copernicus creds).
<code>app/jobs/sse.py</code>	○	SSE progress transport for the async web-service job path (<code>app/jobs</code>); manifest line 40 marks the web service not exercised by a study run.
<code>app/jobs/store.py</code>	○	Job-record persistence seam for the async web-service job path (<code>app/jobs</code>); manifest line 40 marks the web service not exercised by a study run.
<code>app/jobs/worker.py</code>	○	arq worker endpoint for the async web-service job path (optional [<code>jobs</code>] extra, live Redis); manifest line 40 marks the web service not exercised by a study run.
<code>app/models/__init__.py</code>	○	Web-boundary input-model package (<code>app/models</code>); manifest line 40 marks all of <code>app/models</code> not exercised by a study run.
<code>app/models/inputs.py</code>	○	Wizard form→scenario mapping model for the web service (<code>app/models</code>); manifest line 40 marks it not exercised by a study run (the study fed committed scenario YAMLS to the CLI directly).
<code>app/services/__init__.py</code>	○	Service-seam package for the web service (<code>app/services</code>); manifest line 40 marks it not exercised by a study run.
<code>app/services/pipeline_service.py</code>	○	In-memory service seam used only by the web/API layer (<code>app/services</code>); manifest line 40 marks the web service not exercised by a study run (the study used the CLI <code>run_full_pipeline_v14.py</code> path, not this seam).
<code>app/services/report_global_sa.py</code>	○	Serves the synchronous HTTP report route's global-SA section (<code>app/services</code>); manifest line 40 marks the web service not exercised, and manifest line 33 notes standalone <code>global_sa/Morris</code> was not run as a separate feasibility step (the <code>capital_risk_emit</code> report path omits this block).
<code>app/services/report_tornado.py</code>	○	Serves the synchronous HTTP report route's tornado section (<code>app/services</code>); manifest line 40 marks the web service not exercised (the feasibility tornado

Module	Cov	Reason not included
		came from the standalone AEP tornado + capital_risk_emit, which does not recompute this block).
app/web/___init__.py	○	HTMX wizard UI package (app/web); manifest line 40 marks all of app/web not exercised by a study run.
app/web/auth_cookie.py	○	Session gate for the HTMX wizard UI (app/web); manifest line 40 marks the web service not exercised by a study run.
app/web/routes.py	○	Human-facing HTMX wizard routes (app/web); manifest line 40 marks the web service not exercised by a study run.
config/___init__.py	○	Zero-byte package marker; the manifest's 'config/* (if a .py)' bullet places it in the legacy/sandbox/config bucket, and the feasibility run's configuration was driven by Hydra YAML scenarios (scenarios/dutchbay_lendercase_2025Q4.yaml), not this empty package.
dutchbay_bootstrap.py	○	Developer environment-setup helper run by hand, not part of a feasibility study run; manifest lists dutchbay_bootstrap* under legacy/sandbox bootstrap tooling (inferred).
dutchbay_bootstrap_rules.py	○	GWTF ruleset-CSV validation tool for developer/CI governance, not exercised by a wind feasibility run; manifest classes bootstrap/ruleset tooling as non-feasibility (inferred).
examples/ monte_carlo_lender_pack_example.py	○	Manifest lists examples/* under 'Legacy / sandbox / examples' not part of the deliverable; it is an argparse demo script invoked by hand, and the feasibility MC (capital-risk emitter, n_trials=2000) ran via run_full_pipeline_v14.py + app/reports/capital_risk_emit, not through this example.
finance/bess_lcos.py	○	Manifest lists finance/bess_lcos under the not_applicable BESS track; a read-only storage reporting view not imported by the fired wind pipeline.
finance/bess_project_economics.py	○	Manifest lists finance/bess_project_economics under the not_applicable BESS track (CEB distributed-BESS / Kalpitiya BESS tender case); not part of the wind feasibility deliverable and not imported by the fired pipeline.
finance/epc_margin.py	○	Off-cashflow EPC supply-contract (construction margin) model for a BESS/EPC tender case, not the operational wind CFADS path; not imported by the fired pipeline (manifest treats BESS/EPC-tender work as a separate track) (inferred).
legacy/___init__.py	○	Manifest explicitly buckets legacy/* as not_applicable ('Legacy / sandbox / examples'); the package doc states these modules are on no production path (no CLI/pipeline/report/app caller), so the feasibility run never imported it.
legacy/stress_tests_v14.py	○	Quarantined legacy per manifest ('legacy/*') and its own docstring (#473 MC-2/3): reachable from no Hydra CLI, pipeline, report, or FastAPI app, so it did not fire; real tail risk in the run came from analytics/mc + capital_risk_layer_v14, not this module.
legacy_scripts/archive/make_clean_zip.py	○	Manifest buckets legacy_scripts/* under 'Legacy / sandbox / examples' and 'Tooling / non-feasibility scripts'; this is a repo-packaging CLI unrelated to running a wind feasibility study, so it did not fire.
scripts/01_github_repo_scanner.py	○	Repo-inventory/tech-debt tooling, not a feasibility module; manifest excludes non-feasibility scripts/tooling (inferred).
scripts/ FINAL_CORRECTED_sensitivity_v14.py	○	Non-runnable patch-note stub for a sensitivity bug; not a feasibility module (inferred).
scripts/FIX_SENSITIVITY_LINE_489.py	○	Non-runnable code-snippet patch note, not exercised by any run (inferred).
scripts/add_pydantic_v2_compat_stubs.py	○	Historical Pydantic-v2 migration patcher, source-mutating tooling not part of a feasibility run (inferred).
scripts/analysis/analyze_directory.py	○	Manifest not_applicable: scripts/analysis/* is excluded tooling (inferred).
scripts/analysis/gen_scenario_yaml.py	○	Manifest not_applicable: scripts/analysis/* excluded tooling; interactive scenario scaffolder not part of a run (inferred).
scripts/analysis/wacc_engine_yaml.py	○	Explicitly deprecated/superseded standalone WACC calculator under scripts/analysis/* (manifest not_applicable) (inferred).
scripts/build/build_zip_from_manifest.py	○	Manifest not_applicable: scripts/build/* is excluded release tooling (inferred).
scripts/build/generate_manifest.py	○	Manifest not_applicable: scripts/build/* excluded tooling (inferred).
scripts/build/make_essential_zip.py	○	Manifest not_applicable: scripts/build/* excluded tooling (inferred).
scripts/check_fields.py	○	

Module	Cov	Reason not included
		Ad-hoc field-listing helper on a saved results JSON; not part of the feasibility run (inferred).
scripts/ci/check_all_py_files.py	○	Manifest not_applicable: scripts/ci/* is excluded CI tooling (inferred).
scripts/ci/check_legacy_imports.py	○	Manifest not_applicable: scripts/ci/* legacy-import guard, excluded CI tooling (inferred).
scripts/ci/check_staged_py_files.py	○	Manifest not_applicable: scripts/ci/* excluded CI tooling (inferred).
scripts/ci/ci_structure_check.py	○	Manifest not_applicable: scripts/ci/* excluded CI tooling (inferred).
scripts/ci/model_guard.py	○	Manifest not_applicable: scripts/ci/* model-guard, excluded CI/validation tooling (inferred).
scripts/ci/validate.py	○	Manifest not_applicable: scripts/ci/* legacy validation tooling (inferred).
scripts/codebase_datalake_ingress.py	○	Codebase-datalake indexing tooling, not a feasibility module; manifest excludes analysis/build tooling (inferred).
scripts/compile_changelog.py	○	Release/changelog tooling, not part of a study run (inferred).
scripts/datalake_refresh_and_diff.py	○	Codebase-snapshot/diff tooling, not a feasibility module (inferred).
scripts/dutchbay_cleanup_analyzer.py	○	Repo-cleanup inventory tooling, not exercised by a feasibility run (inferred).
scripts/dutchbay_manifest_builder.py	○	Repo-manifest builder tooling, not a feasibility module (inferred).
scripts/fix_metrics_typing.py	○	Historical mypy-fix source patcher, not part of a run (inferred).
scripts/fix_yaml_depreciation_method.py	○	Historical YAML-migration patcher, not exercised by a feasibility run (inferred).
scripts/gen_architecture_diagram.py	○	Docs/architecture-diagram generation tooling, not a feasibility module (inferred).
scripts/generate_solar_assessment_report.py	○	Solar single-tech PDF report generator; the deliverable is a WIND feasibility study, and solar only appeared as scenarios in the fired tech_comparison, not via this standalone solar-PDF script (inferred).
scripts/github/auto_close_issues.py	○	Manifest not_applicable: scripts/github/* is excluded GitHub automation tooling (inferred).
scripts/github/gh_tools.py	○	Manifest not_applicable: scripts/github/* excluded GitHub tooling (inferred).
scripts/go_with_the_flow_ci.py	○	Local CI/lint orchestration tooling; manifest excludes scripts/ci/* and CI helpers (inferred).
scripts/inspect_era5.py	○	Ad-hoc NetCDF structure inspector against a hardcoded local file, not part of any feasibility run (inferred).
scripts/instrument_statutory_debug.py	○	Ad-hoc debug-instrumentation helper, not part of a feasibility run (inferred).
scripts/legacy_runners/run_complete_analysis_fixed.py	○	Manifest not_applicable: scripts/legacy_runners/* excluded legacy runners (inferred).
scripts/legacy_runners/run_wind_download_v14.py	○	Manifest not_applicable: scripts/legacy_runners/* excluded; the fired wind download used era5_retrieval (ARCO), not this legacy CDS runner (inferred).
scripts/make_clean_zip.py	○	Release-archive build tooling; manifest excludes scripts/build/* (inferred).
scripts/patch_contracts_v14_add_tornado_single.py	○	Historical contracts-file source patcher, not part of a run (inferred).
scripts/process_era5_wind_data.py	○	Superseded Sprint-10 NetCDF preprocessing; the fired wind path used wind_resource/era5_retrieval (ARCO), not this ad-hoc NetCDF extractor (manifest wind-stack fired list) (inferred).
scripts/provision_web_secrets.py	○	Web-service (#788) secret-provisioning tooling; manifest classifies the web productization layer as not_applicable (inferred).
scripts/quarantine_bad_irr_mc_tests.py	○	Test-maintenance tooling, pure test-support excluded by the manifest (inferred).
scripts/research/__init__.py	○	Manifest not_applicable: scripts/research/* excluded research tooling; standalone V12 model independent of the v14 engine (inferred).
scripts/research/charts.py	○	Manifest not_applicable: scripts/research/* excluded research tooling (inferred).
scripts/research/legacy_v12.py	○	Manifest not_applicable: scripts/research/* standalone legacy V12 model, superseded by the v14 engine (inferred).
scripts/research/optimization.py	○	Manifest not_applicable: scripts/research/* standalone V12 optimiser, independent of the v14 engine (inferred).

Module	Cov	Reason not included
scripts/run_epc_margin.py	○	EPC construction-margin path is a separate deal type from the operational v14 CFADS engine; not part of the wind feasibility deliverable (finance.epc_margin fired as a library import per manifest, but this EPC-deal CLI did not) (inferred).
scripts/run_full_pipeline_sprint12.py	○	Superseded Sprint-12 orchestrator, not the v14 canonical path used for the deliverable; manifest excludes 'most of scripts/*' except the two v14 runners (inferred).
scripts/validate_pysam_offline.py	○	Offline PySAM sandbox validation with explicit NO PIPELINE INTEGRATION; manifest lists pysam_sandbox as sandbox/not_applicable (inferred).
scripts/verify_complete.py	○	Ad-hoc post-hoc DSCR-fix verification reading a committed out/*.json file; not part of producing the feasibility deliverable (inferred).
scripts/verify_dscr.py	○	Ad-hoc DSCR inspection reading a committed out/*.json artifact; not exercised by the study run (inferred).